

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week-1

Week 1:

A local library is currently managing its books and members using a physical notebook, which has resulted in several operational problems such as misplaced books, difficulty tracking borrowed items, and inaccurate member records. To modernize its operations, the library plans to develop a database system called **BookFlow**. As a database developer, your task is to create the SQL foundation for this system. First, create a new database named **bookflow_db**. Then, design two tables: a **Books** table containing the columns **book_id**, **title**, **isbn**, and **published_year**, and a **Members** table containing the columns **member_id**, **full_name**, and **email**. Ensure data integrity by setting **book_id** and **member_id** as **PRIMARY KEYS**, applying a **NOT NULL** constraint to the **title** column so that every book must have a name, enforcing **UNIQUE** constraints on both **isbn** and **email** to prevent duplicate entries, and adding a **CHECK** constraint to ensure that the **published_year** is not in the future (for example, it must be less than 2027). Finally, insert three sample records into both the **Books** and **Members** tables to verify that the database structure and constraints are functioning correctly. This exercise will help you develop skills in schema definition, data integrity enforcement, and the use of SQL Data Definition Language (DDL) commands such as **CREATE DATABASE**, **CREATE TABLE**, and **INSERT** statements.

Solutions:

1. The Scenario

A local library currently tracks its books and members in a physical notebook. Books are being lost, and there is no way to know who has which book. The goal is to build the SQL foundation for "BookFlow" so that every book has a unique ISBN and no member can sign up without a valid (unique) email.

The requirements are:

- Create a database named **bookflow_db**.
- Create a books table (**book_id**, **title**, **isbn** **UNIQUE**, **published_year**) and a members table (**member_id**, **full_name**, **email** **UNIQUE**).
- Enforce **PRIMARY KEY** on both IDs, **NOT NULL** on title, and a **CHECK** constraint so **published_year** is not in the future (< 2027).
- Seed 3 books and 3 members, then test that the rules actually reject bad data.

2. Step 1 Database Setup

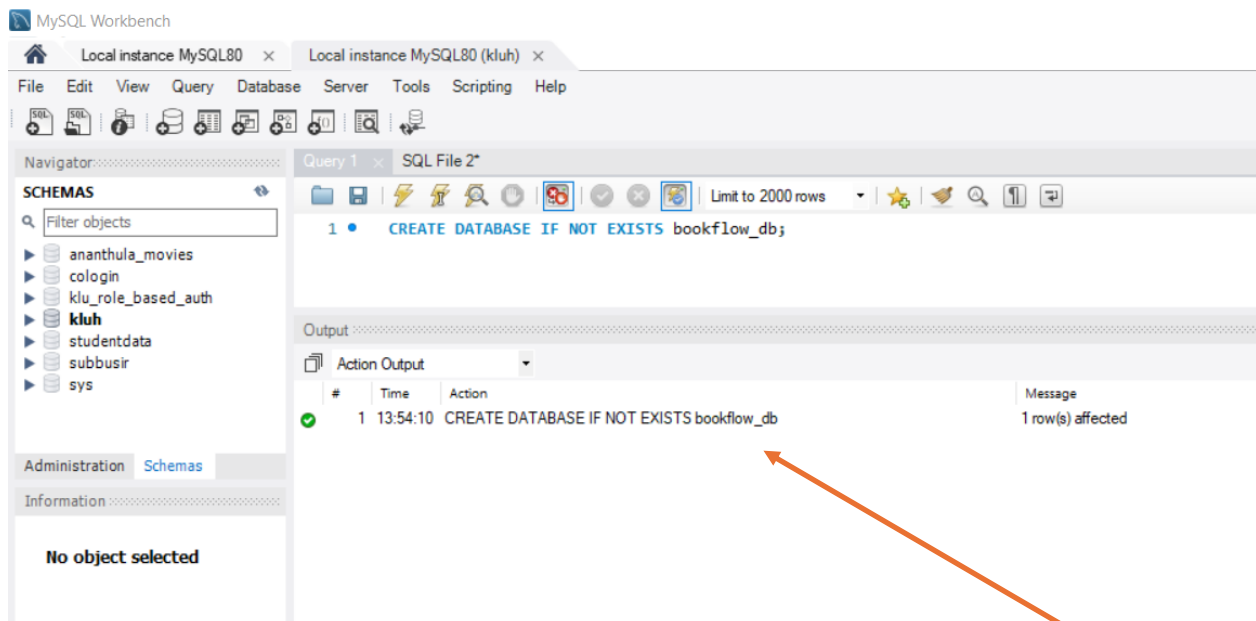
Query 1: Create the database

```
CREATE DATABASE IF NOT EXISTS bookflow_db;
```

Explanation:

CREATE DATABASE creates a new, empty database (a container for tables). The IF NOT EXISTS clause makes the command safe to re-run: if bookflow_db already exists, MySQL skips creation instead of throwing an error. This is a DDL (Data Definition Language) command.

Output:

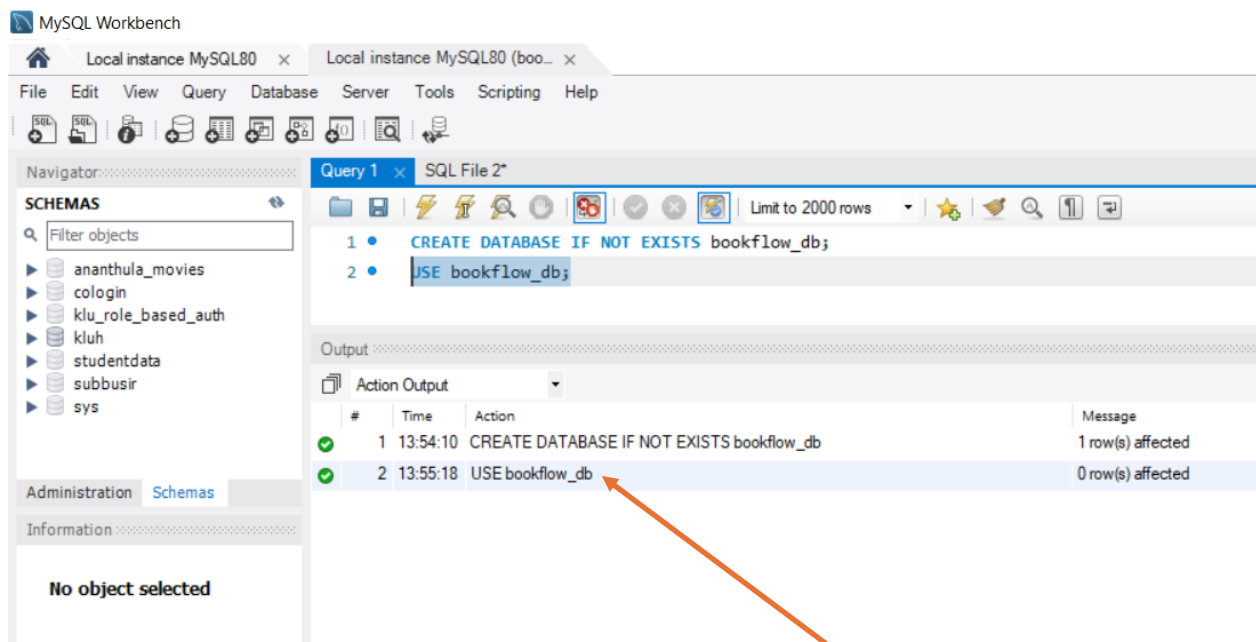


Query 2: Select the database

```
USE bookflow_db;
```

Explanation:

USE switches the current session to bookflow_db so that every subsequent CREATE TABLE, INSERT, and SELECT runs inside this database. Without this step, the tables would be created in whatever database was previously active.



Output:

Database changed

3. Step 2 Table Creation (DDL) with Constraints

Query 3: Create the books table

```
CREATE TABLE books (  
  book_id    INT AUTO_INCREMENT PRIMARY KEY,  
  title      VARCHAR(255) NOT NULL,  
  isbn       VARCHAR(13) NOT NULL UNIQUE,  
  published_year INT,  
  CONSTRAINT chk_published_year CHECK (published_year < 2027)  
);
```

Explanation:

Line-by-line breakdown of every column and constraint:

- `book_id INT AUTO_INCREMENT PRIMARY KEY` — the PRIMARY KEY uniquely identifies each row; it is automatically NOT NULL and UNIQUE. AUTO_INCREMENT makes MySQL generate 1, 2, 3, ... automatically, so we never insert the ID manually.
- `title VARCHAR(255) NOT NULL` — NOT NULL means a book cannot be saved without a name. An INSERT with a NULL title is rejected.
- `isbn VARCHAR(13) NOT NULL UNIQUE` — UNIQUE guarantees no two books share the same ISBN. VARCHAR is used instead of INT because ISBNs can have leading zeros and ISBN-10 can end in the letter X.
- `published_year INT` with `CHECK (published_year < 2027)` — the named CHECK constraint (`chk_published_year`) rejects any year of 2027 or later, preventing "future" books from entering the system.

The screenshot shows a SQL IDE interface. On the left is a 'Navigator' pane with a 'SCHEMAS' section containing a tree of database objects like 'ananthula_movies', 'cologin', 'klu_role_based_auth', 'kluh', 'studentdata', 'subbusir', 'sys', 'Tables', 'Views', and 'Stored Procedures'. Below this is a 'No object selected' message. The main editor pane shows a SQL script for 'Query 1' in 'SQL File 2*'. The script consists of three lines: 1. 'CREATE DATABASE IF NOT EXISTS bookflow_db;', 2. 'USE bookflow_db;', and 3. 'CREATE TABLE books ('. The table definition includes columns: 'book_id' (INT AUTO_INCREMENT PRIMARY KEY), 'title' (VARCHAR(255) NOT NULL), 'isbn' (VARCHAR(13) NOT NULL UNIQUE), 'published_year' (INT), and a check constraint 'CONSTRAINT chk_published_year CHECK (published_year < 2027)'. The script ends with ');'. Below the editor is an 'Output' pane with a table showing the execution results. The table has columns for '#', 'Time', 'Action', and 'Message'. It shows three rows of execution: 1. '13:54:10 CREATE DATABASE IF NOT EXISTS bookflow_db' with '1 row(s) affected', 2. '13:55:18 USE bookflow_db' with '0 row(s) affected', and 3. '13:56:52 CREATE TABLE books (' with '0 row(s) affected'. An orange arrow points from the 'CREATE TABLE books (' line in the script to the third row in the output table.

```
1 • CREATE DATABASE IF NOT EXISTS bookflow_db;
2 • USE bookflow_db;
3 • CREATE TABLE books (
4     book_id      INT AUTO_INCREMENT PRIMARY KEY,
5     title        VARCHAR(255) NOT NULL,
6     isbn         VARCHAR(13) NOT NULL UNIQUE,
7     published_year INT,
8     CONSTRAINT chk_published_year CHECK (published_year < 2027)
9 );
10
```

#	Time	Action	Message
✓ 1	13:54:10	CREATE DATABASE IF NOT EXISTS bookflow_db	1 row(s) affected
✓ 2	13:55:18	USE bookflow_db	0 row(s) affected
✓ 3	13:56:52	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected

Query 4: Create the members table

```
CREATE TABLE members (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE
);
```

Explanation:

- `member_id INT AUTO_INCREMENT PRIMARY KEY` — unique identity for every member, generated automatically.
- `full_name VARCHAR(100) NOT NULL` — a member must have a name on record.
- `email VARCHAR(150) NOT NULL UNIQUE` — `NOT NULL` means no signup without an email; `UNIQUE` means one account per email address. Together these satisfy the "no member can sign up without a valid email" rule.

Output:

Query 1 x SQL File 2*

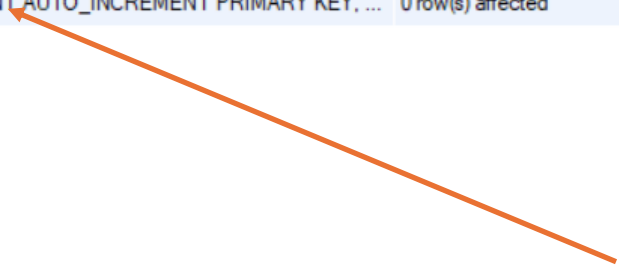
Limit to 2000 rows

```
2 • USE bookflow_db;
3 • CREATE TABLE books (
4     book_id      INT AUTO_INCREMENT PRIMARY KEY,
5     title         VARCHAR(255) NOT NULL,
6     isbn          VARCHAR(13) NOT NULL UNIQUE,
7     published_year INT,
8     CONSTRAINT chk_published_year CHECK (published_year < 2027)
9 );
10
11 • CREATE TABLE members (
12     member_id INT AUTO_INCREMENT PRIMARY KEY,
13     full_name  VARCHAR(100) NOT NULL,
14     email      VARCHAR(150) NOT NULL UNIQUE
15 );
```

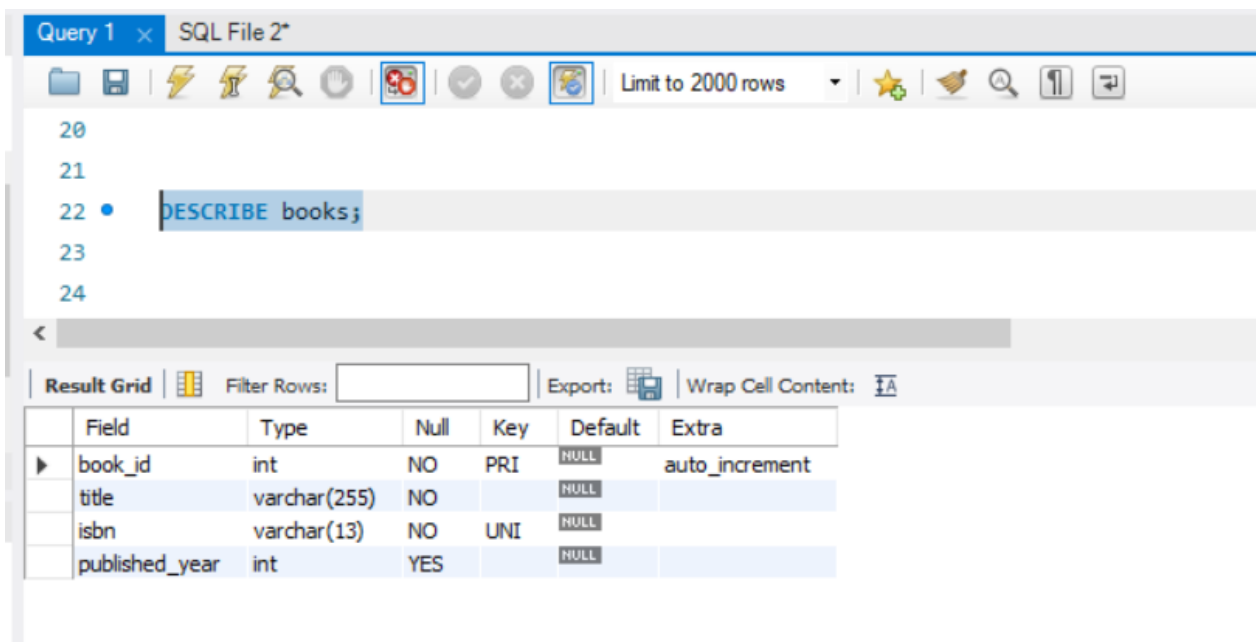
Output

Action Output

#	Time	Action	Message
✓ 1	13:54:10	CREATE DATABASE IF NOT EXISTS bookflow_db	1 row(s) affected
✓ 2	13:55:18	USE bookflow_db	0 row(s) affected
✓ 3	13:56:52	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected
✓ 4	13:57:58	CREATE TABLE members (member_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected



Query 5 (verification): Inspect the table structures

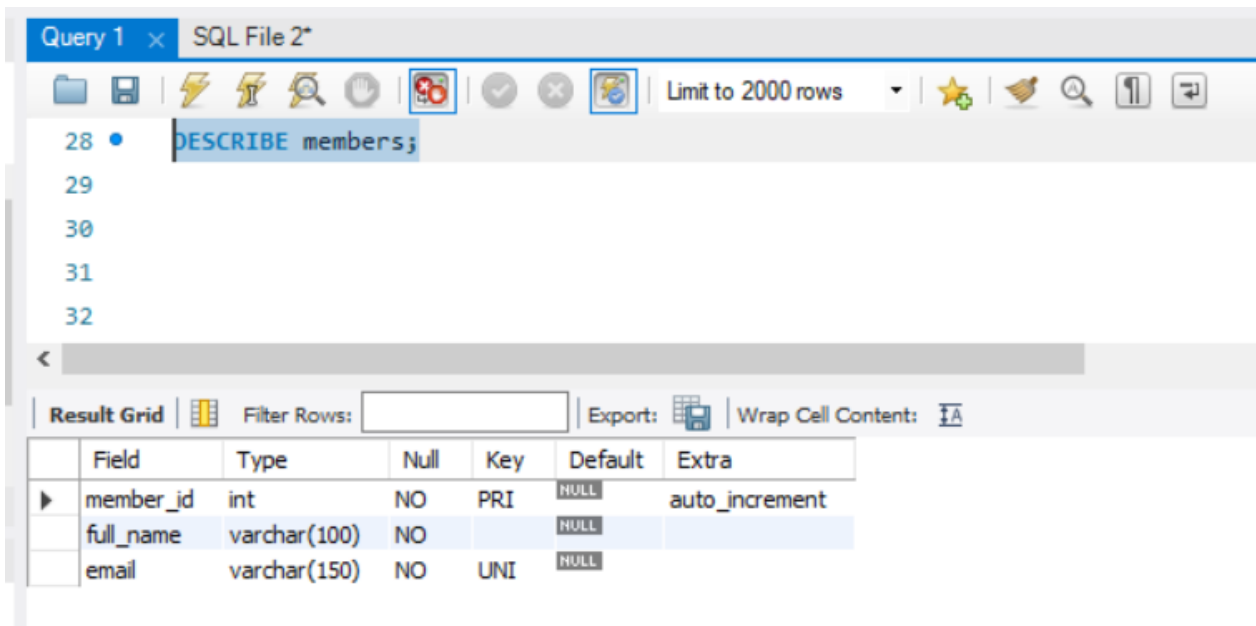


The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the text `DESCRIBE books;`. The result grid displays the schema for the `books` table.

Field	Type	Null	Key	Default	Extra
book_id	int	NO	PRI	NULL	auto_increment
title	varchar(255)	NO		NULL	
isbn	varchar(13)	NO	UNI	NULL	
published_year	int	YES		NULL	

DESCRIBE members;

Output:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the text `DESCRIBE members;`. The result grid displays the schema for the `members` table.

Field	Type	Null	Key	Default	Extra
member_id	int	NO	PRI	NULL	auto_increment
full_name	varchar(100)	NO		NULL	
email	varchar(150)	NO	UNI	NULL	

Explanation:

DESCRIBE shows the schema. Key = PRI confirms the PRIMARY KEY; Key = UNI confirms the UNIQUE constraint on isbn and email; Null = NO confirms NOT NULL columns. This is how an examiner verifies the constraints exist.

Step 3 Initial Seeding (Valid Data)

Query 6: Insert 3 books

```
INSERT INTO books (title, isbn, published_year) VALUES
('The Alchemist', '9780061122415', 1988),
('Clean Code', '9780132350884', 2008),
('Atomic Habits', '9780735211292', 2018);
```

Explanation:

A multi-row INSERT adds three books in a single statement. `book_id` is deliberately omitted — `AUTO_INCREMENT` assigns 1, 2, 3. All three rows pass every constraint: titles are present, ISBNs are unique, and all years are below 2027.

Output:

```
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Query 7: Insert 3 members

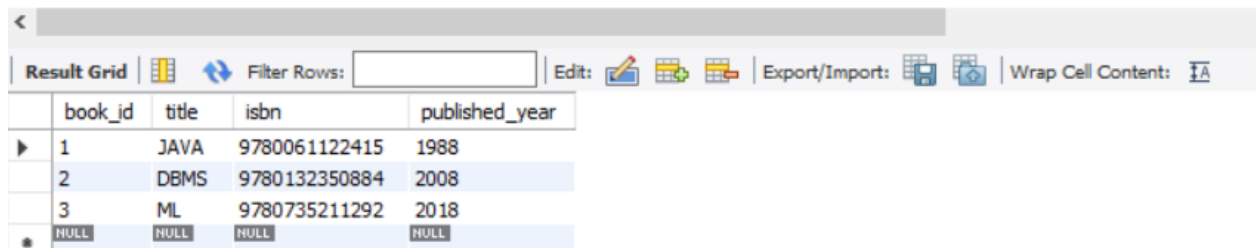
```
INSERT INTO members (full_name, email) VALUES
('Anil Kumar', 'anil.kumar@example.com'),
('Priya Sharma', 'priya.sharma@example.com'),
('Ravi Verma', 'ravi.verma@example.com');
```

Explanation:

Three members are registered, each with a distinct email, so the `UNIQUE` constraint is satisfied. `member_id` values 1–3 are generated automatically.

Output:

```
30 • INSERT INTO books (title, isbn, published_year) VALUES
31   ('JAVA', '9780061122415', 1988),
32   ('DBMS', '9780132350884', 2008),
33   ('ML', '9780735211292', 2018);
34
35 • SELECT * FROM books;
36
```



	book_id	title	isbn	published_year
▶	1	JAVA	9780061122415	1988
	2	DBMS	9780132350884	2008
	3	ML	9780735211292	2018
*	NULL	NULL	NULL	NULL

Query 1 x SQL File 2*

Limit to 2000 rows

```
39 • INSERT INTO members (full_name, email) VALUES
40 ('SUBBU ROY', 'subbusir@example.com'),
41 ('SATEESH', 'sateesh@example.com'),
42 ('KRISHNA', 'krishna@example.com');
43
44 • SELECT * FROM members;
45
```

Result Grid

	member_id	full_name	email
▶	1	SUBBU ROY	subbusir@example.com
	2	SATEESH	sateesh@example.com
	3	KRISHNA	krishna@example.com
*	NULL	NULL	NULL

SELECT * retrieves every column and row. Both tables show exactly 3 rows with auto-generated IDs, confirming the seeding succeeded.

5. Step 4 Constraint Tests (These MUST Fail)

Each query below deliberately violates one rule. A correct schema rejects all four. In an exam, showing the error message is the proof that data integrity works.

Test 1: Duplicate ISBN (UNIQUE violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES ('Fake Copy', '9780061122415', 2000);
```

Output:

51	•	INSERT INTO books (title, isbn, published_year)
52		VALUES ('Fake Copy', '9780061122415', 2000);

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	13:54:10	CREATE DATABASE IF NOT EXISTS bookflow_db	1 row(s) affected	0.016 sec
✓ 2	13:55:18	USE bookflow_db	0 row(s) affected	0.000 sec
✓ 3	13:56:52	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.094 sec
✓ 4	13:57:58	CREATE TABLE members (member_id INT AUTO_INCREMENT PRIMARY KEY...	0 row(s) affected	0.062 sec
✓ 5	13:58:44	DESCRIBE books	4 row(s) returned	0.000 sec / 0.000 sec
✓ 6	13:59:49	DESCRIBE members	3 row(s) returned	0.016 sec / 0.000 sec
✗ 7	14:02:39	INSERT INTO books (title, isbn, published_year) VALUES (JAVA', '9780061122415',...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corre...	0.015 sec
✓ 8	14:05:16	INSERT INTO books (title, isbn, published_year) VALUES (JAVA', '9780061122415'...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
✓ 9	14:05:46	SELECT * FROM books LIMIT 0, 2000	3 row(s) returned	0.000 sec / 0.000 sec
✓ 10	14:08:20	INSERT INTO members (full_name, email) VALUES ('SUBBU ROY', 'subbusir@exa...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
✓ 11	14:08:51	SELECT * FROM members LIMIT 0, 2000	3 row(s) returned	0.016 sec / 0.000 sec
✗ 12	14:10:18	INSERT INTO books (title, isbn, published_year) VALUES ('Fake Copy', '9780061122...	Error Code: 1062. Duplicate entry '9780061122415' for key 'books.isbn'	0.015 sec

Explanation:

The ISBN 9780061122415 already belongs to 'The Alchemist'. The UNIQUE constraint on isbn rejects the row, so two physical records can never claim the same ISBN.

Test 2: NULL title (NOT NULL violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES (NULL, '99999999999999', 2010);
```

Output:

55	
56	
57	
58	• <code>INSERT INTO books (title, isbn, published_year)</code>
59	<code>VALUES (NULL, '999999999999', 2010);</code>
60	

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	13:54:10	CREATE DATABASE IF NOT EXISTS bookflow_db	1 row(s) affected	0.016 sec
✓ 2	13:55:18	USE bookflow_db	0 row(s) affected	0.000 sec
✓ 3	13:56:52	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.094 sec
✓ 4	13:57:58	CREATE TABLE members (member_id INT AUTO_INCREMENT PRIMARY KEY...	0 row(s) affected	0.062 sec
✓ 5	13:58:44	DESCRIBE books	4 row(s) returned	0.000 sec / 0.000 sec
✓ 6	13:59:49	DESCRIBE members	3 row(s) returned	0.016 sec / 0.000 sec
✗ 7	14:02:39	INSERT INTO books (title, isbn, published_year) VALUES (JAVA', '9780061122415'...	Error Code: 1064. You have an error in your SQL syntax; check the manual that come...	0.015 sec
✓ 8	14:05:16	INSERT INTO books (title, isbn, published_year) VALUES (JAVA', '9780061122415'...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
✓ 9	14:05:46	SELECT * FROM books LIMIT 0, 2000	3 row(s) returned	0.000 sec / 0.000 sec
✓ 10	14:08:20	INSERT INTO members (full_name, email) VALUES ('SUBBU ROY', 'subbusir@exa...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
✓ 11	14:08:51	SELECT * FROM members LIMIT 0, 2000	3 row(s) returned	0.016 sec / 0.000 sec
✗ 12	14:10:18	INSERT INTO books (title, isbn, published_year) VALUES ('Fake Copy', '9780061122...	Error Code: 1062. Duplicate entry '9780061122415' for key 'books.isbn'	0.015 sec
✗ 13	14:11:54	INSERT INTO books (title, isbn, published_year) VALUES (NULL, '999999999999'...	Error Code: 1048. Column 'title' cannot be null	0.000 sec

Explanation:

The NOT NULL constraint fires because a book must have a name. The database refuses to store a nameless book, exactly as the requirement demands.

Test 3: Future publication year (CHECK violation)

```
INSERT INTO books (title, isbn, published_year)
VALUES ('Time Traveler', '88888888888888', 2030);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'chk_published_year' is violated.
```

Explanation:

2030 fails the condition `published_year < 2027`, so the named CHECK constraint `chk_published_year` blocks the insert. Garbage dates cannot enter the table.

Test 4: Duplicate email (UNIQUE violation)

```
INSERT INTO members (full_name, email)
VALUES ('Anil Clone', 'anil.kumar@example.com');
```

Output:

```
ERROR 1062 (23000): Duplicate entry 'anil.kumar@example.com' for key 'members.email'
```

Explanation:

`anil.kumar@example.com` is already registered to member_id 1, so the UNIQUE constraint on email rejects the second signup. One email = one account.

6. Quick Revision Constraint Cheat Sheet

Constraint	Applied On	Purpose	Error If Violated
PRIMARY KEY	book_id, member_id	Unique row identity; NOT NULL + UNIQUE combined	1062 Duplicate entry
NOT NULL	title, full_name, email, isbn	Column must always have a value	1048 Column cannot be null
UNIQUE	isbn, email	No two rows may share the value	1062 Duplicate entry
CHECK	published_year < 2027	Value must satisfy a condition	3819 Check constraint violated
AUTO_INCREMENT	book_id, member_id	Auto-generates sequential IDs	— (not a rule, a convenience)

7. Skills Demonstrated

- Schema Definition — real-world objects (Books, Members) mapped to tables with correct data types (INT for IDs and years, VARCHAR for text and ISBN).
- Data Integrity — UNIQUE, NOT NULL, and CHECK constraints proven to reject garbage data with four failing test inserts.
- DDL Proficiency — CREATE DATABASE, CREATE TABLE, DESCRIBE, and multi-row INSERT executed and verified end to end.

8. How to Run (Practical Exam Steps)

```
# 1. Open MySQL Work bench   Download link: https://dev.mysql.com/downloads/workbench/
# 2. Run queries 1-8 in order, then the 4 failing tests
# 3. Show DESCRIBE + SELECT outputs and the 4 error messages as proof
```

How to Install SQL Workbench For MySQL on Windows

MySQL Workbench is a graphical tool used to interact with MySQL databases through a user-friendly interface.

- It supports data modeling and database design.
- It provides server configuration and user management features.

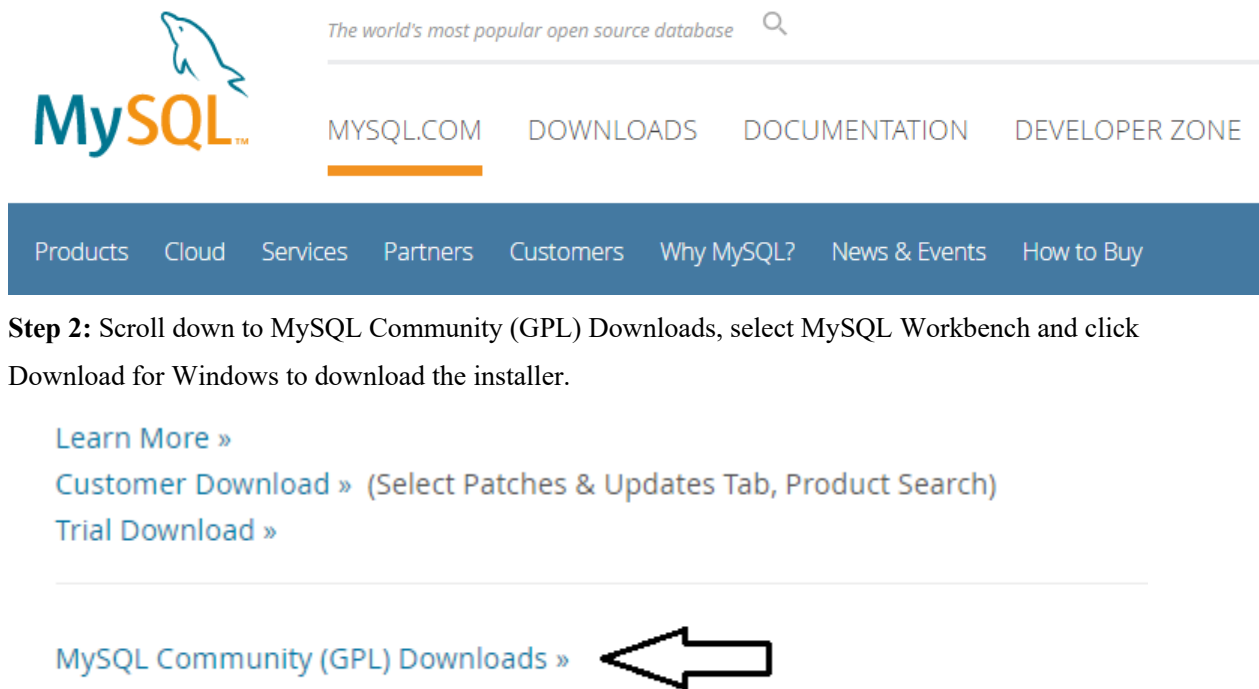
Prerequisites

- Windows 10 or Windows 11 (64-bit)
- Microsoft Visual C++ Redistributable 2019 or later
- .NET Framework 4.5.2 or later
- Approximately 500 MB of free disk space

Method 1: Install via MySQL Installer

This method uses the official MySQL Installer to automatically download and install MySQL Workbench along with any required components on your Windows system.

Step 1: Open your preferred web browser and navigate to the official MySQL website official MySQL Website.



Step 3: On the MySQL Workbench download page, click the first Download link to download the MySQL Installer for Windows.

- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL Cluster
- MySQL Router
- MySQL Shell
- MySQL Workbench
- MySQL Installer for Windows ←
- MySQL for Visual Studio
- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

General Availability (GA) Releases
Archives

MySQL Installer 8.0.27

Select Operating System:

Microsoft Windows ▼

Looking for previous GA versions? ↓

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.27.1.msi)	8.0.27	2.3M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.27.1.msi)	8.0.27	470.2M	Download

MD5: 44b7f3e4c1bdcc641621cf3aa31ea18f4 | Signature

MD5: 9b7af5c91139659b10b84b1ca357d08f | Signature

! We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

Step 4: On the next page, click No thanks, just start my download to begin downloading the installer.

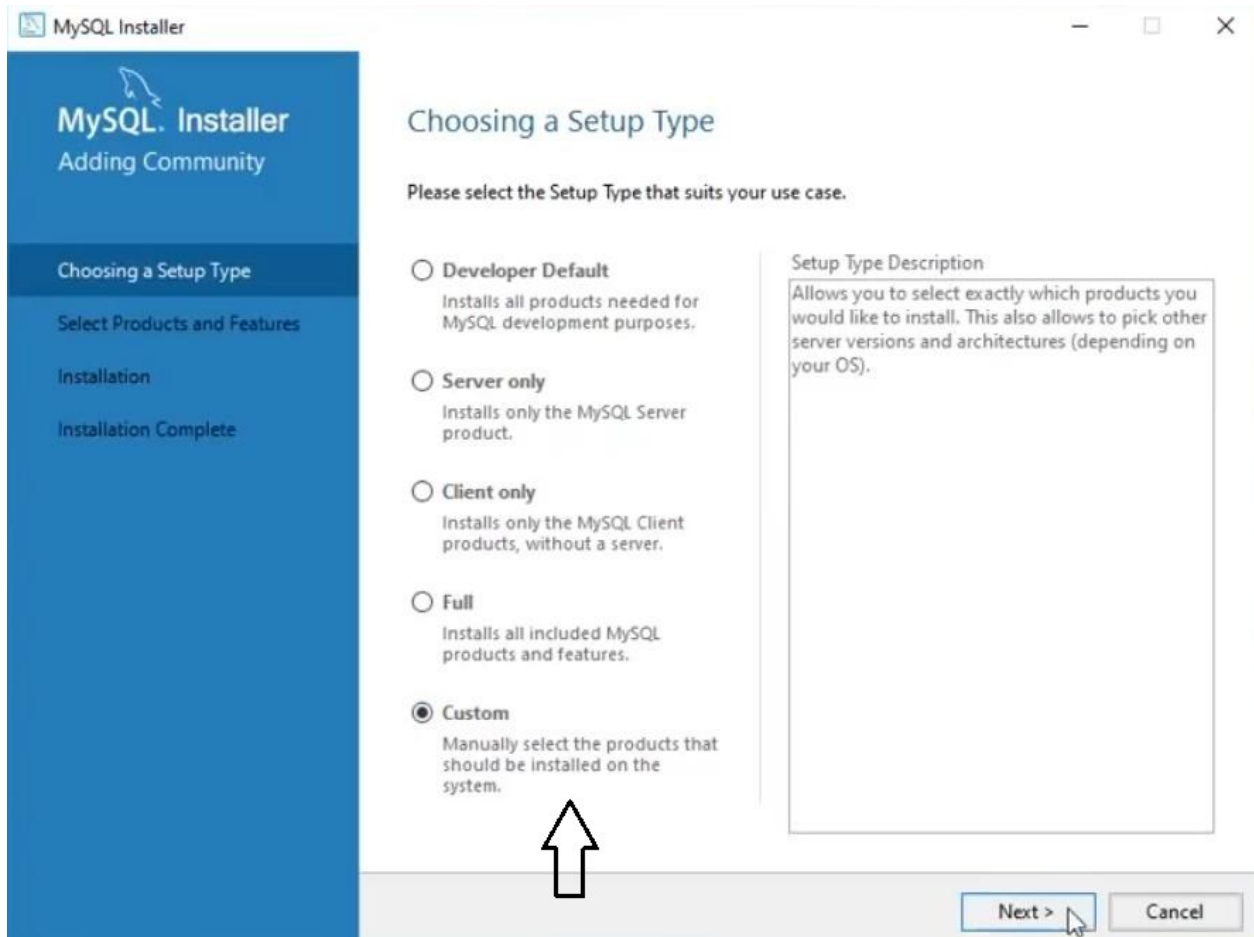
Login »
using my Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you have an account, click the Login link. Otherwise, you can sign up for a free account or view the instructions.

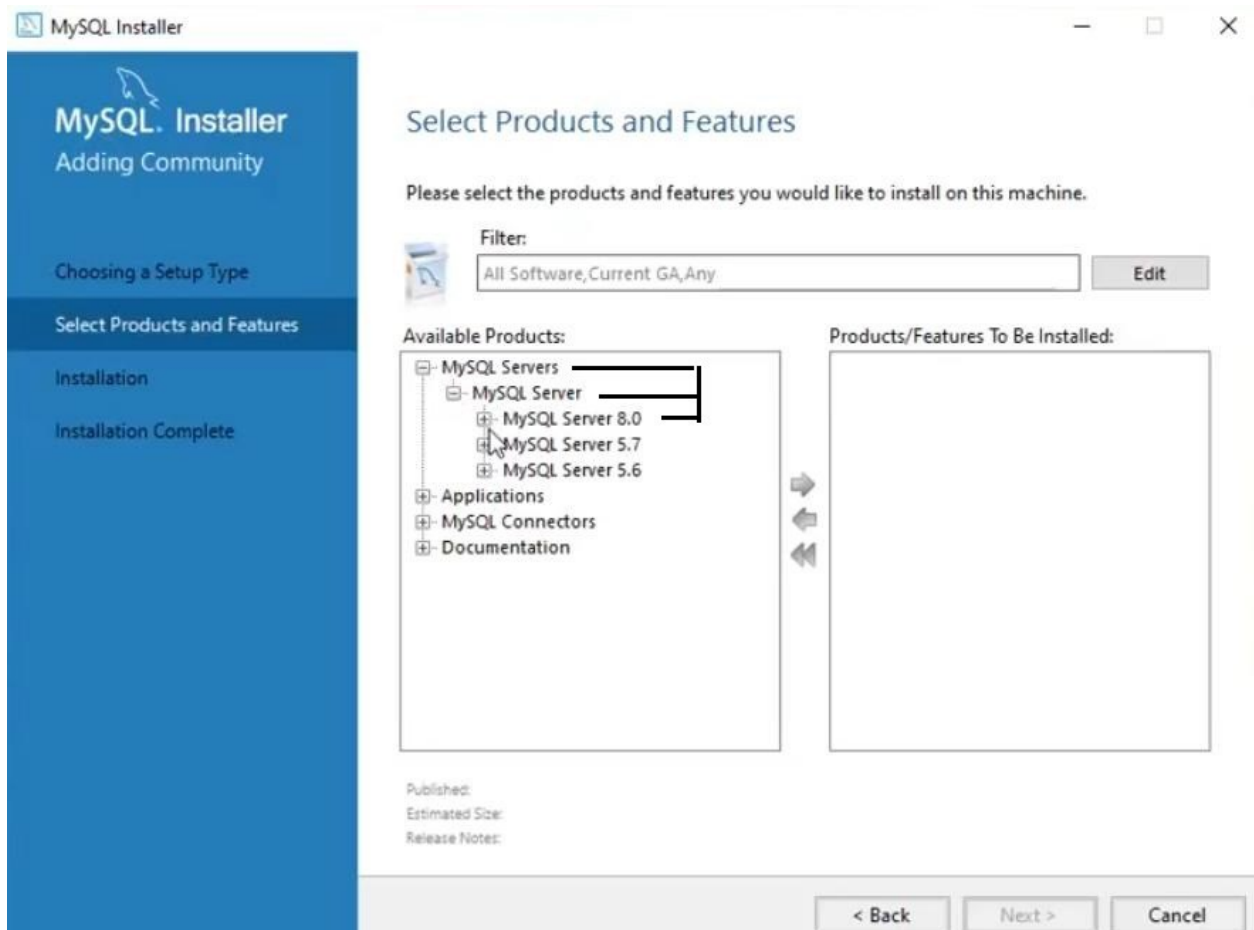
No thanks, just start my download. ←

- After the download is complete, locate the MySQL Installer file (usually in your Downloads folder).
- Double-click the installer file to start the installation process.

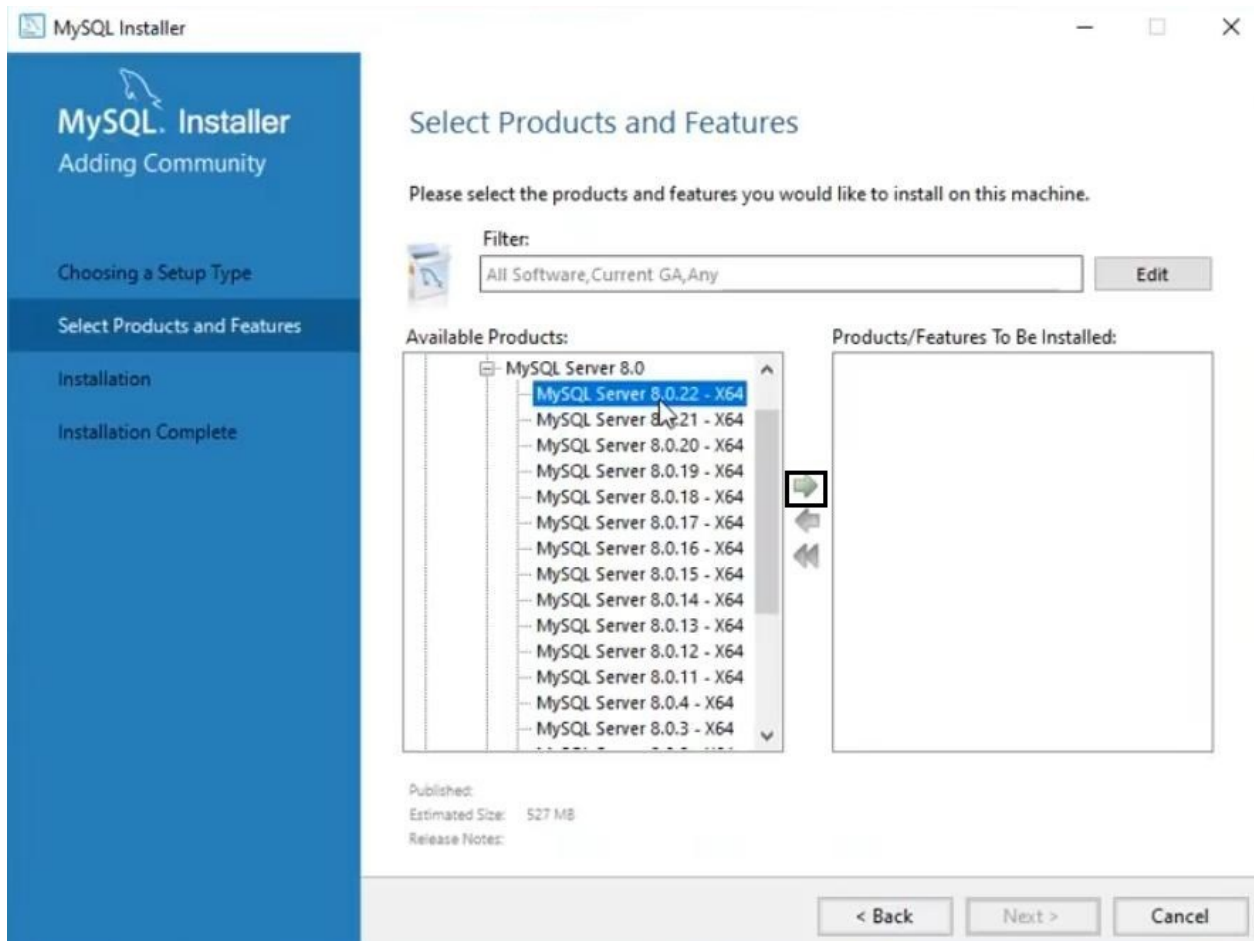
Step 5: The MySQL Installer window will open. Click on Custom to select individual components.



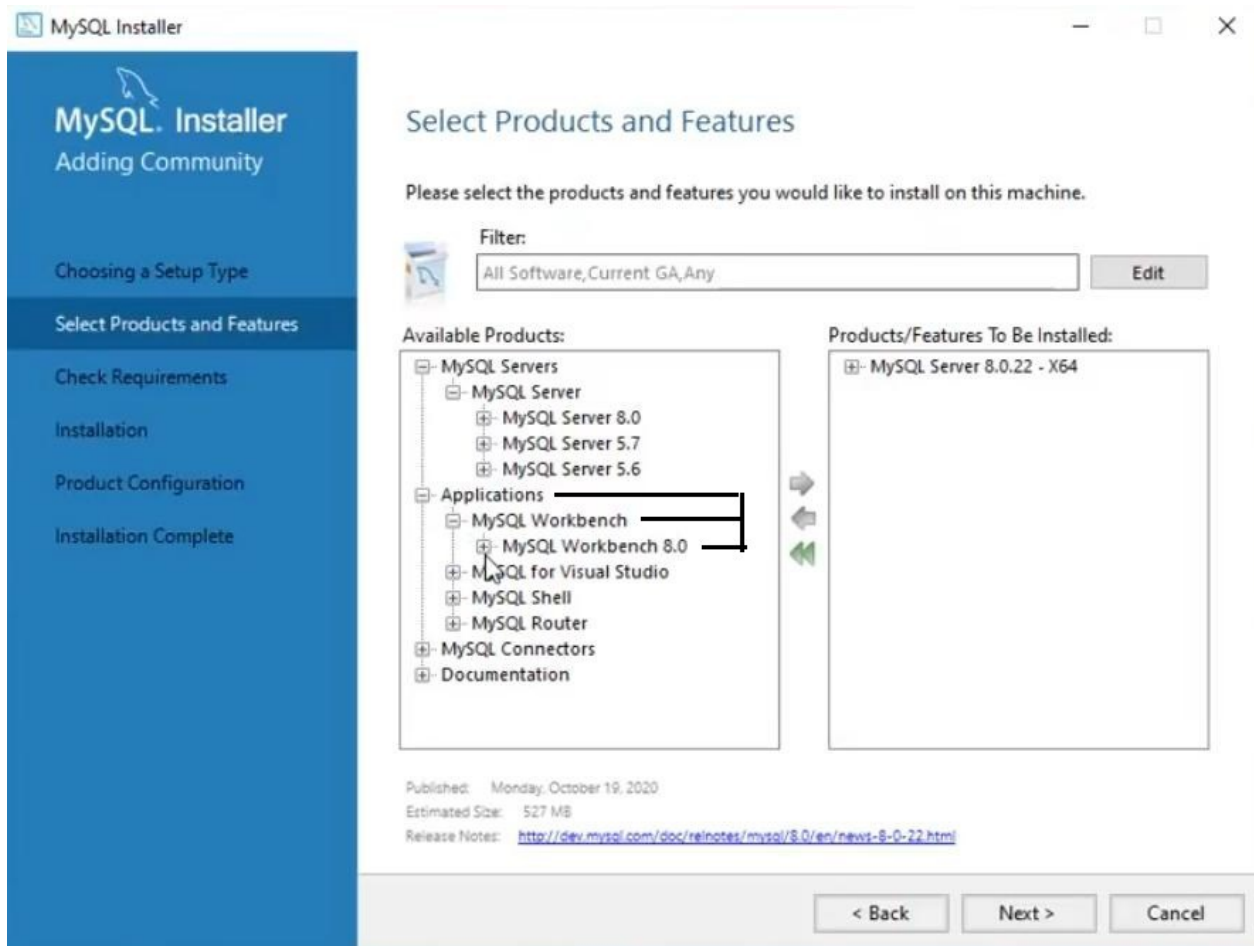
Step 6: In the Select Products and Features section, locate MySQL Workbench and then expand MySQL Server 8.0.



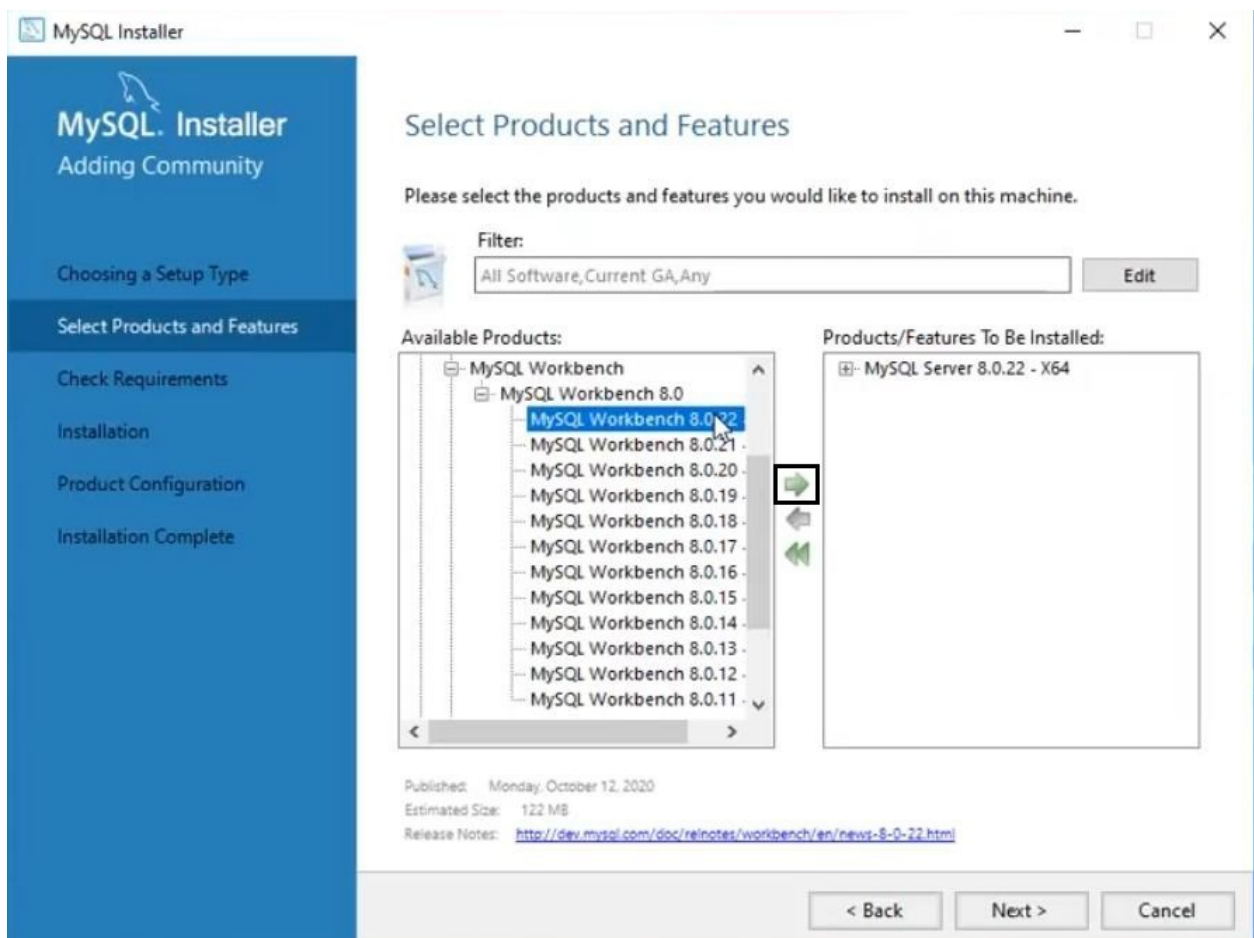
Step 7: Then double click on the latest version & click on the drag button.



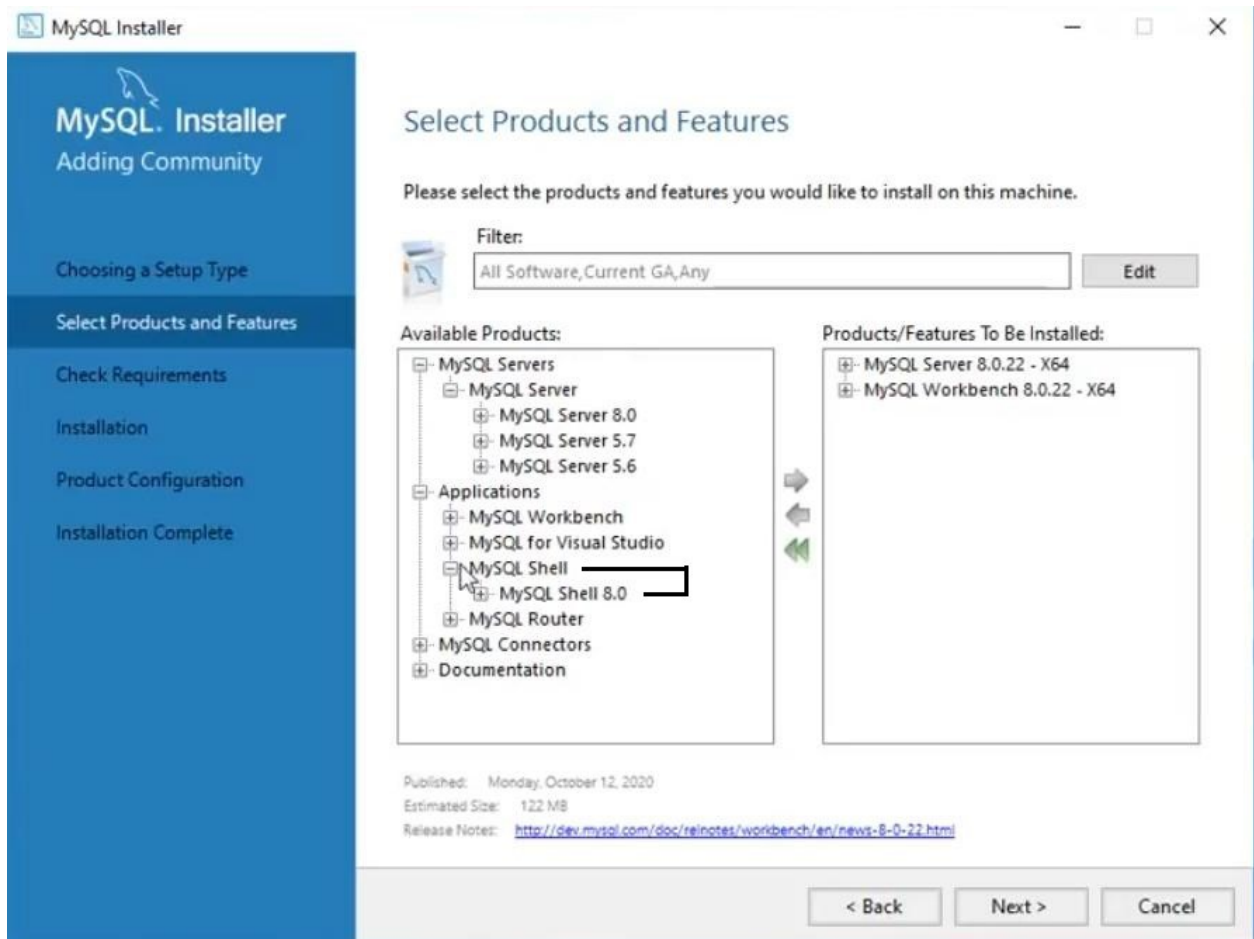
Step 8: Then expand Applications & Then expand MySQL Workbench 8.0



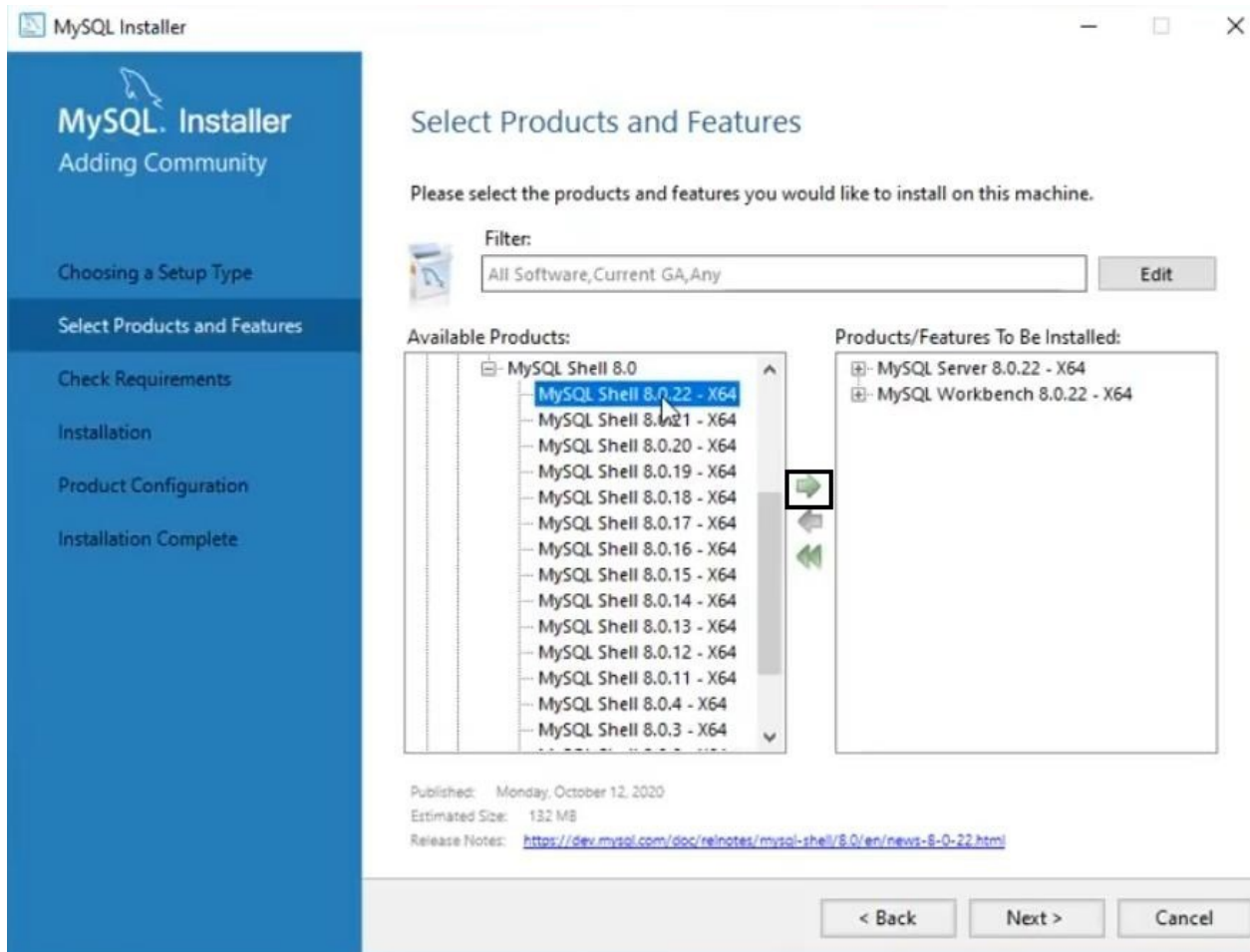
Step 9: Then double click on the latest version & click on the drag button.



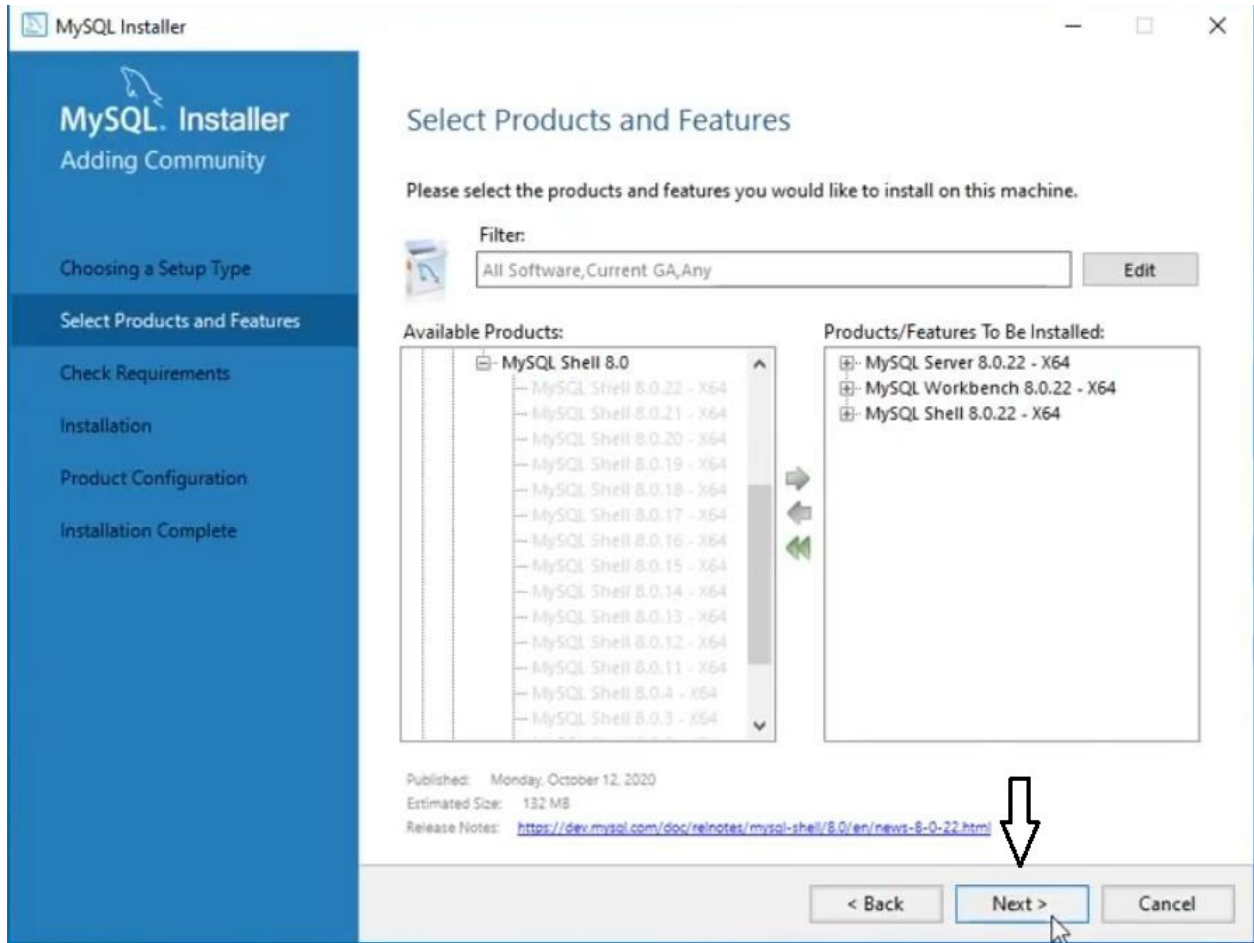
Step 10: Then expand MySQL Shell & then expand MySQL Shell 8.0



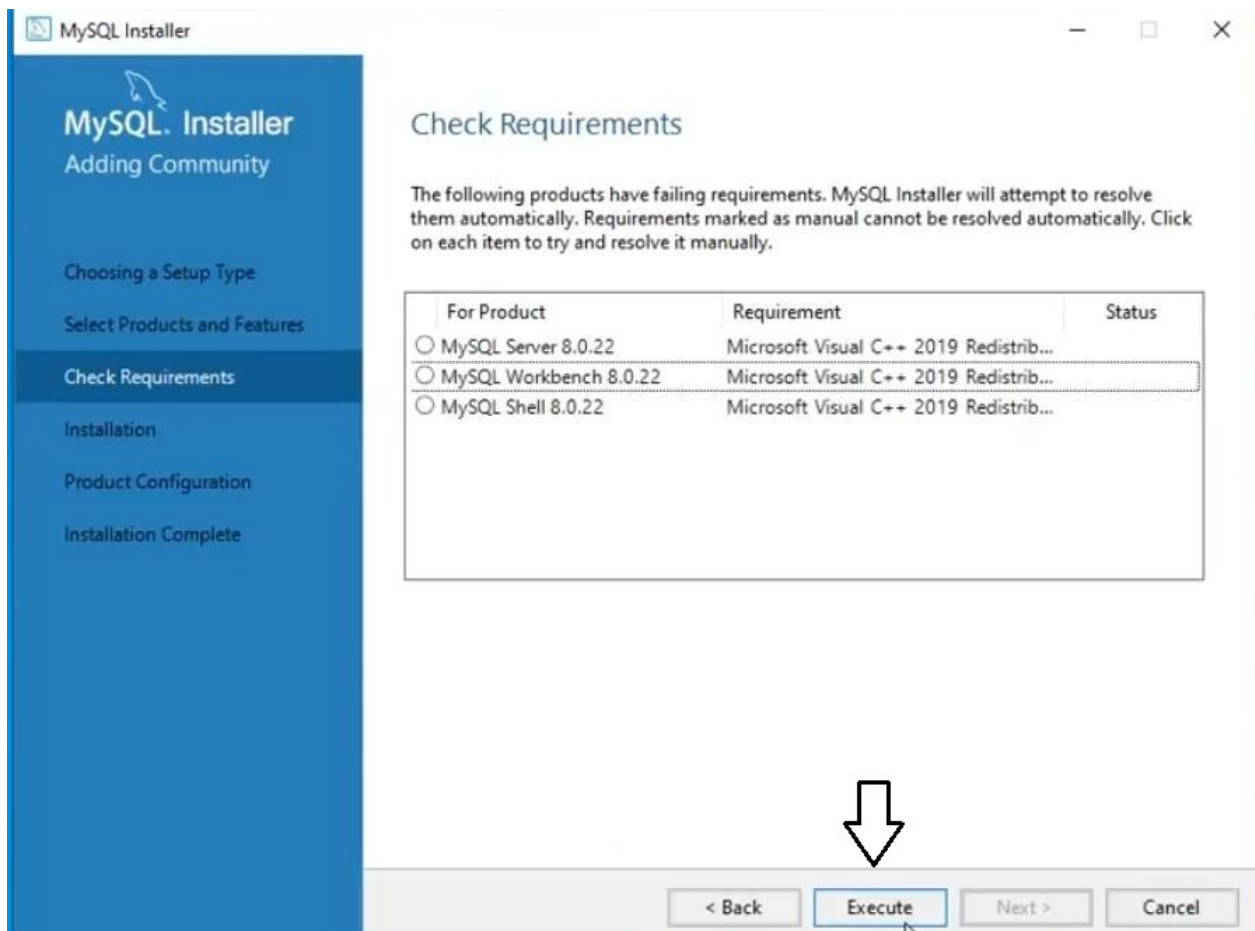
Step 11: Then double click on the latest version & click on the drag button.



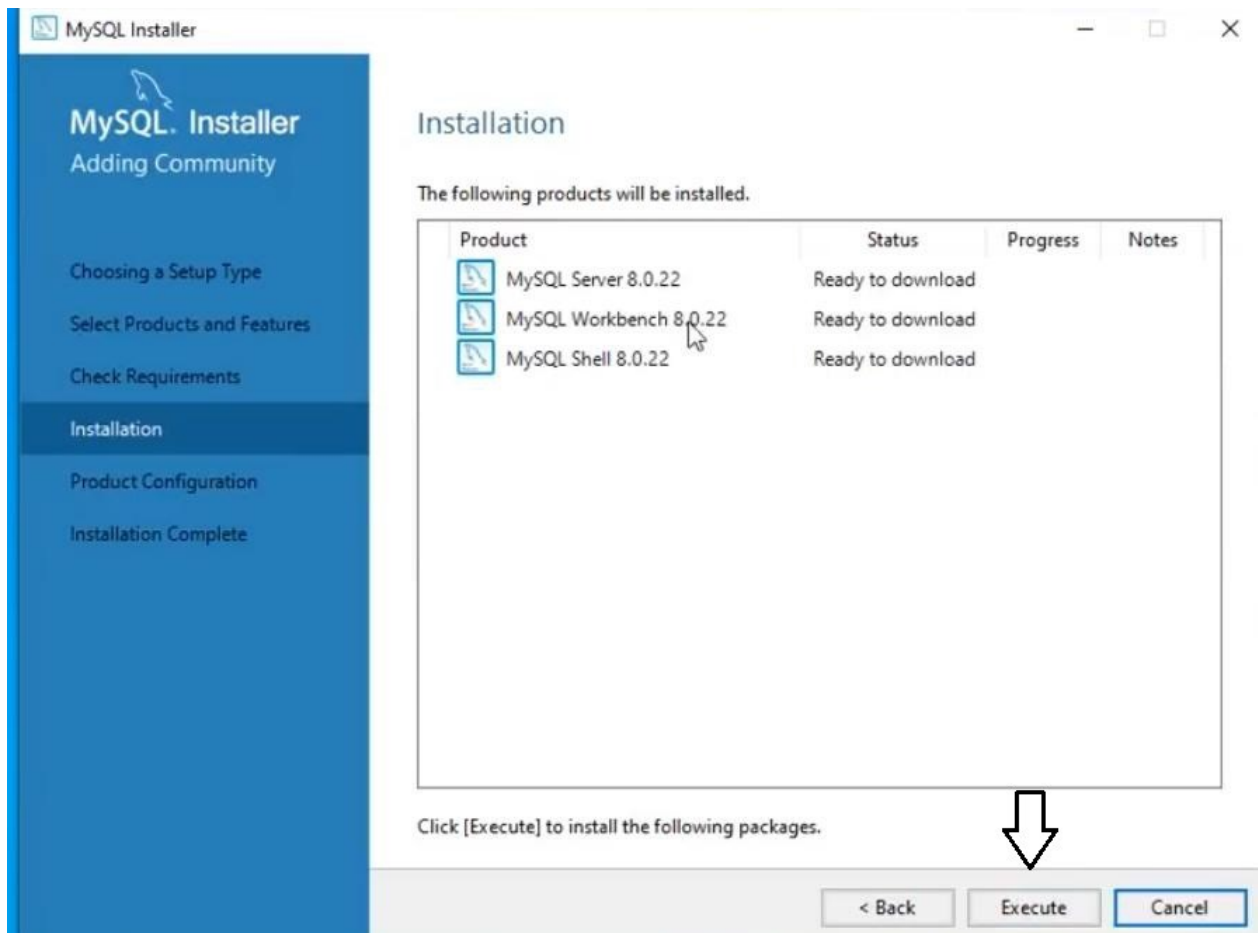
Step 12: Click on Next.



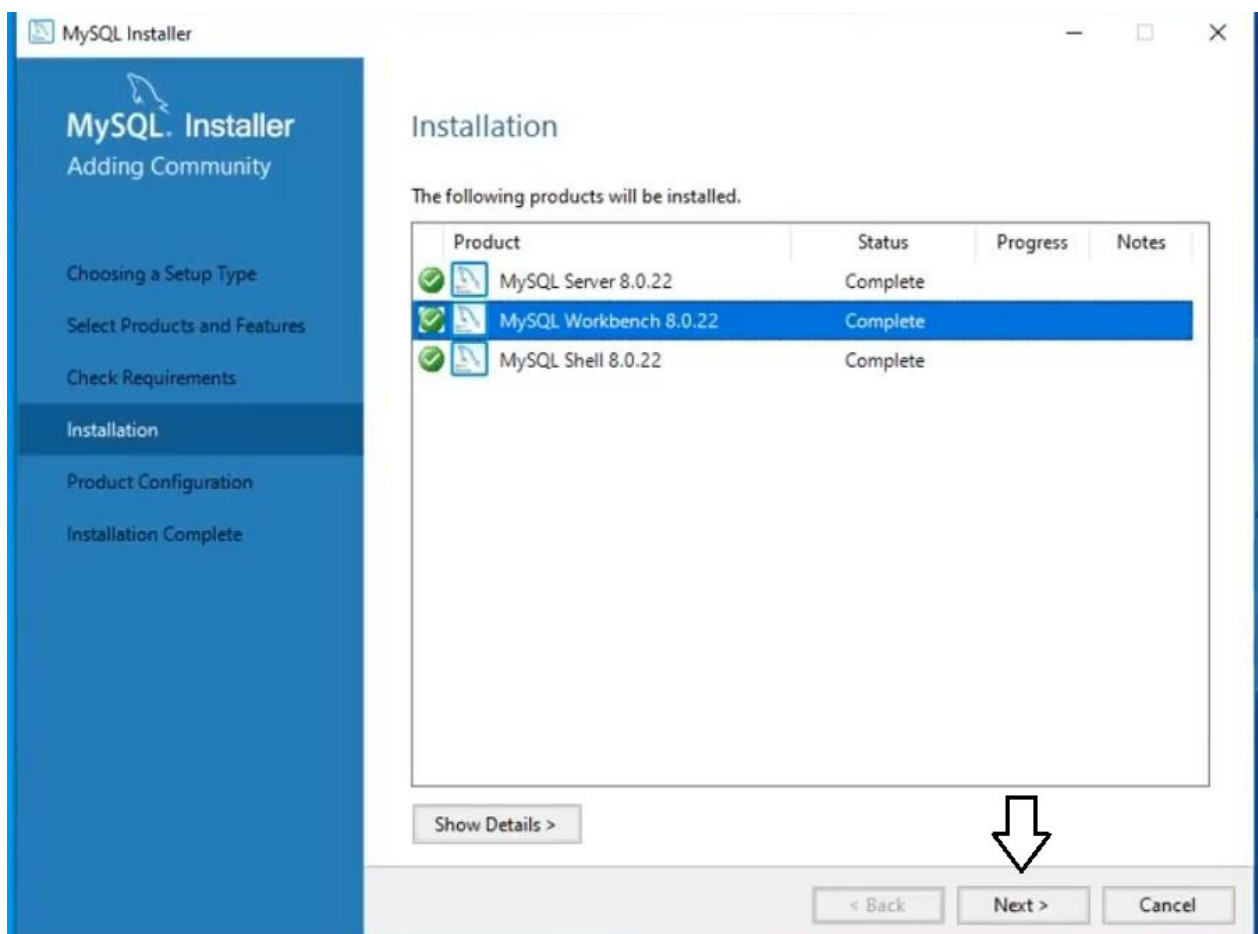
Step 13: After selecting all components, click Execute to begin the installation.



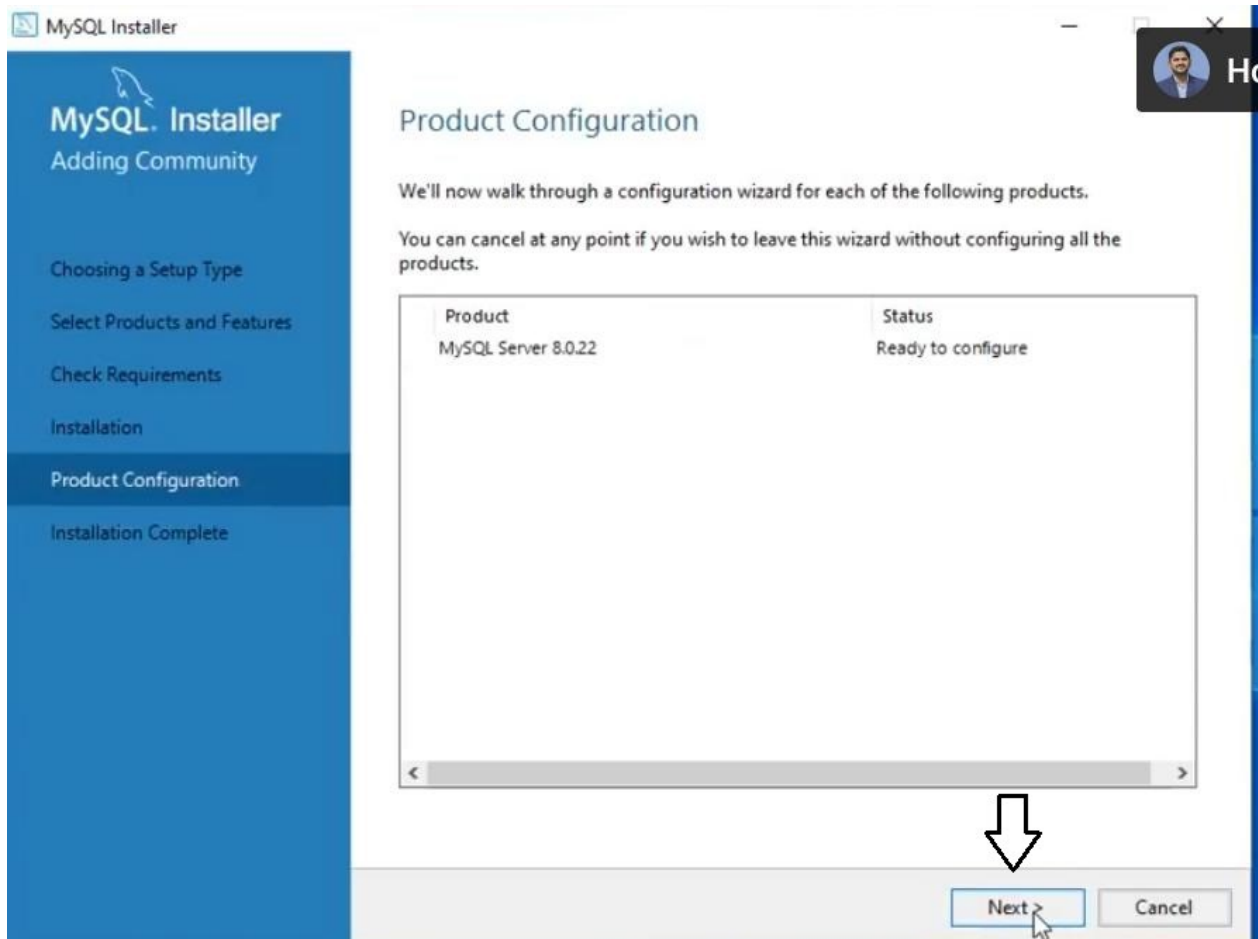
Step 14: After selecting all components, click Execute to begin the installation. MySQL Installer will begin downloading and installing the selected components. This may take a few minutes, depending on your internet speed.



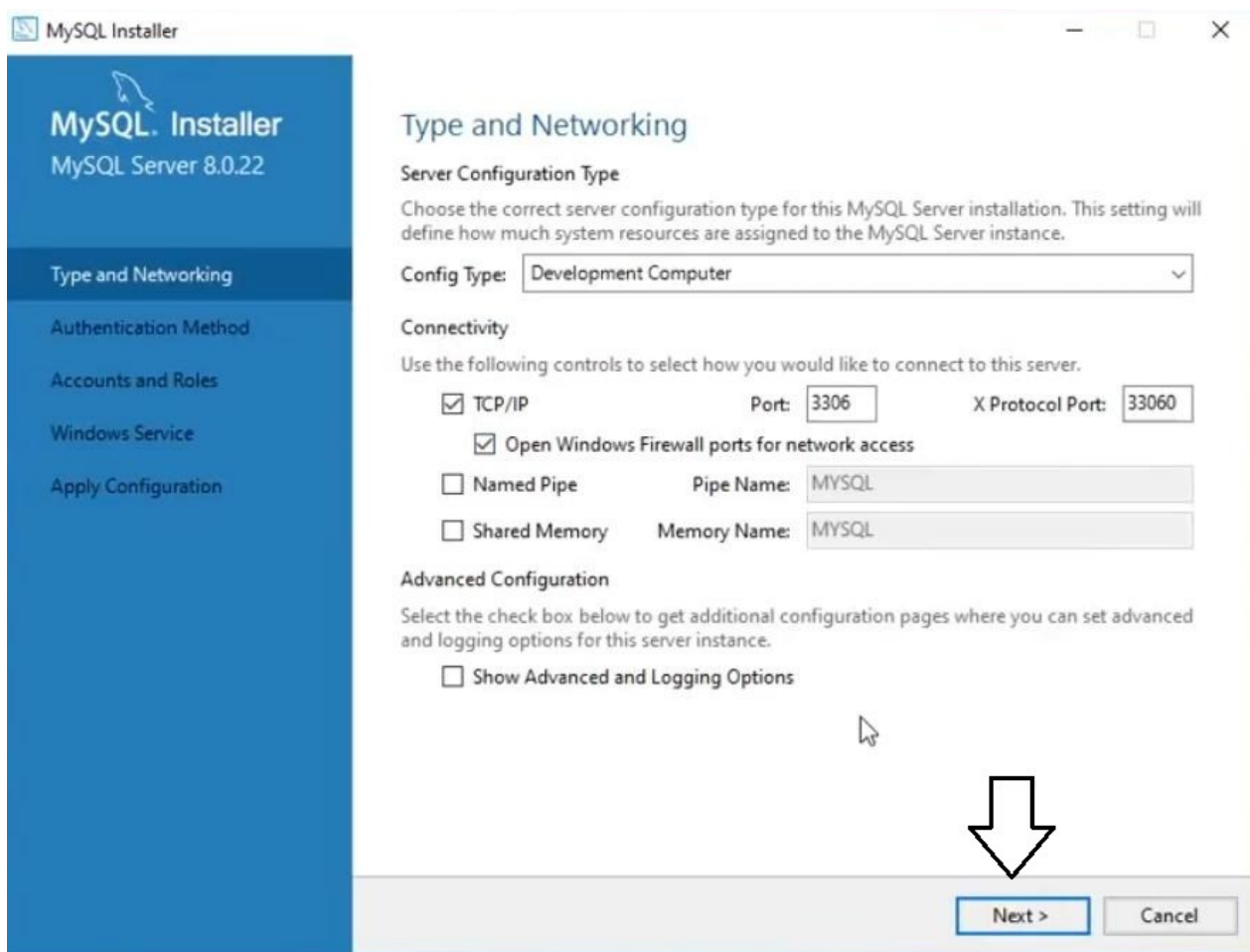
Step 15: Once the installation is complete, click Next.



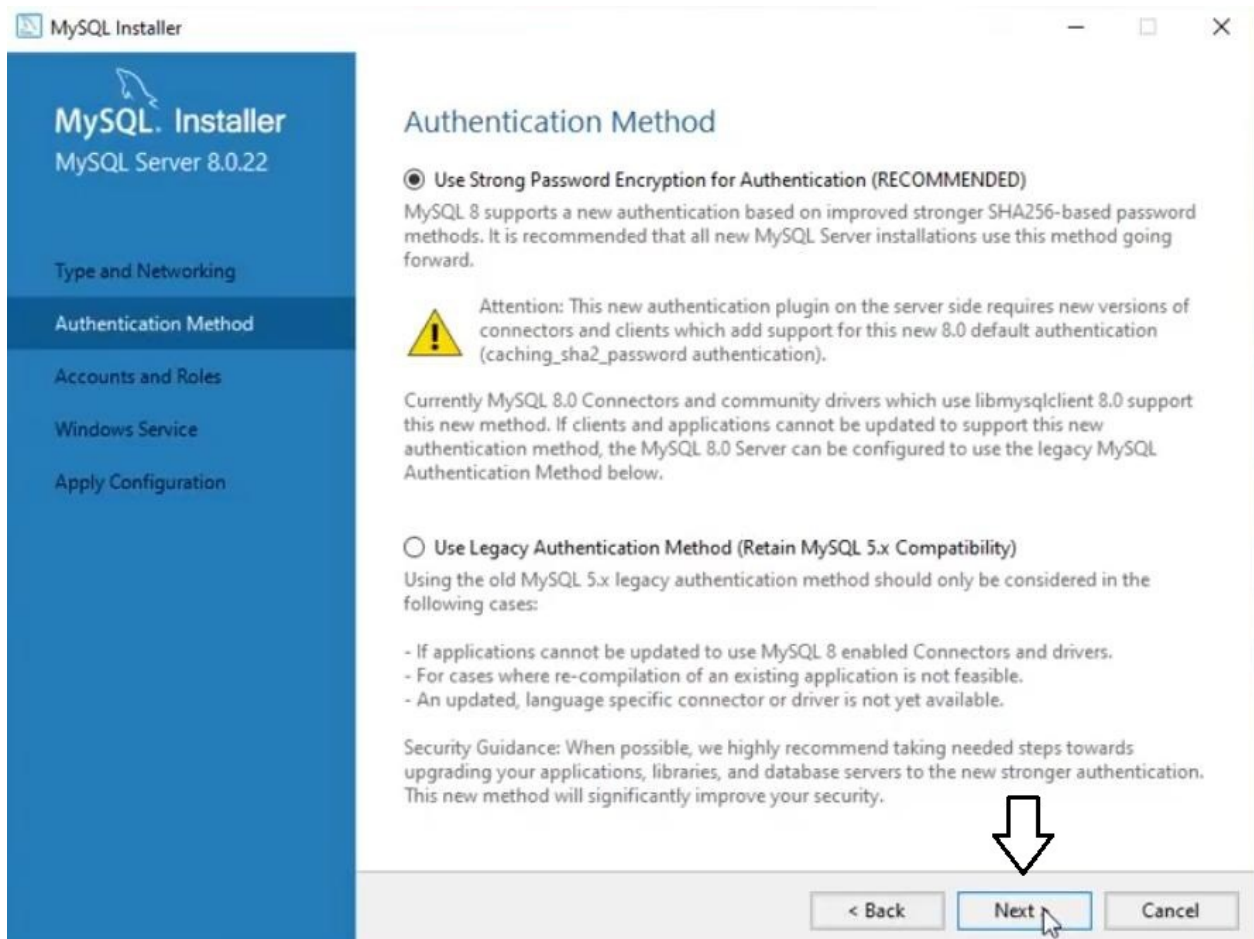
Step 16: Click on Next.



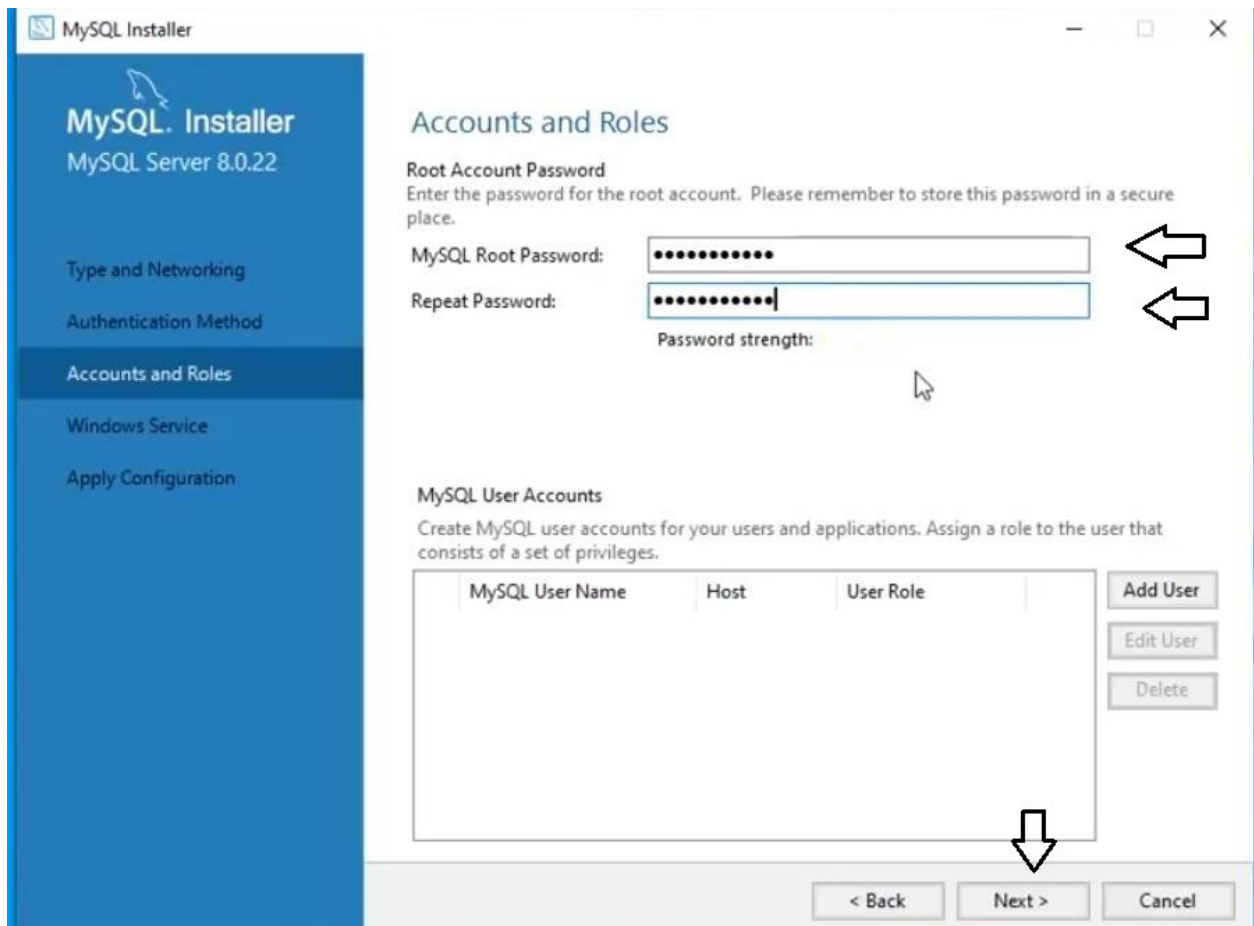
Step 17: Click on Next



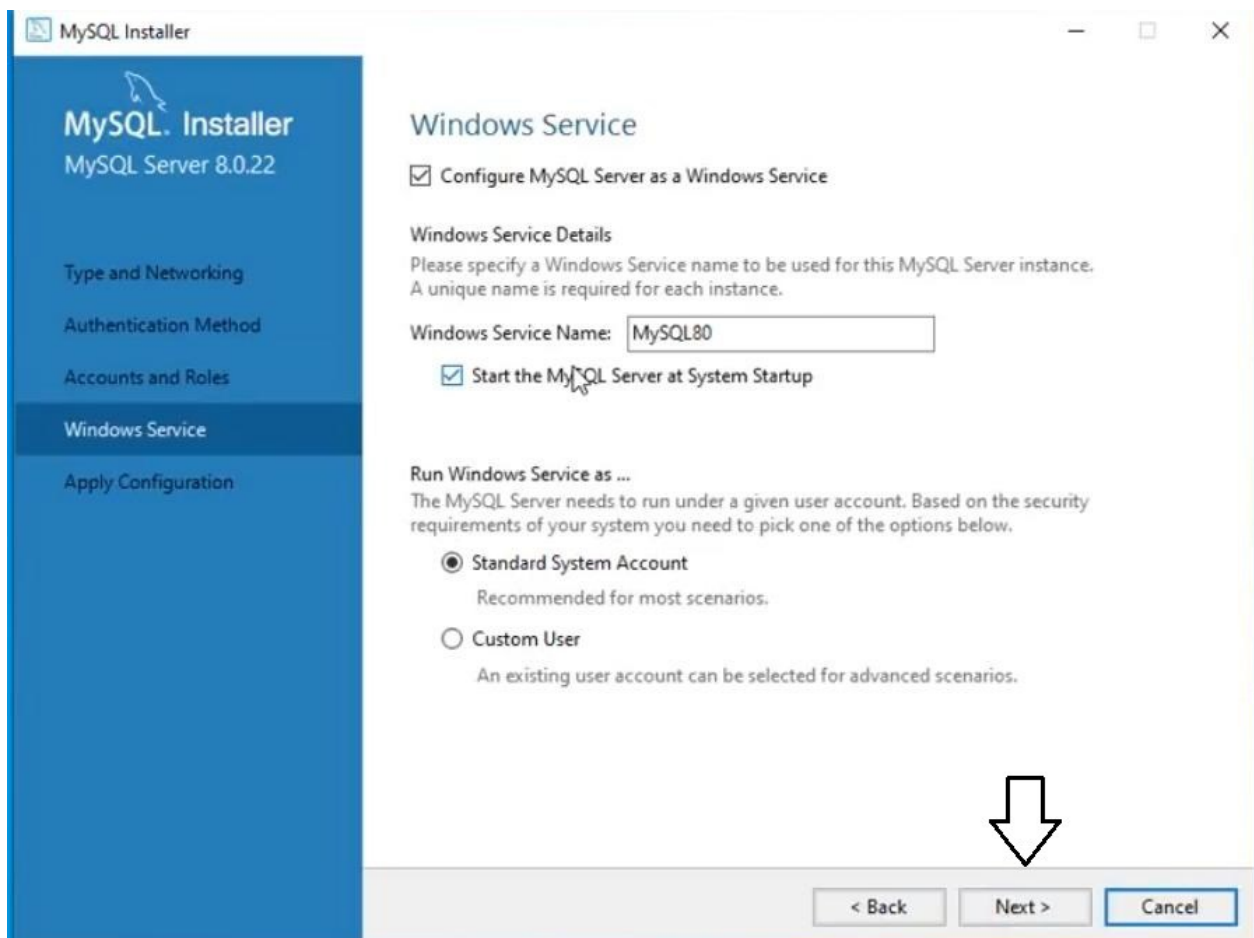
Step 18: Click on Next.



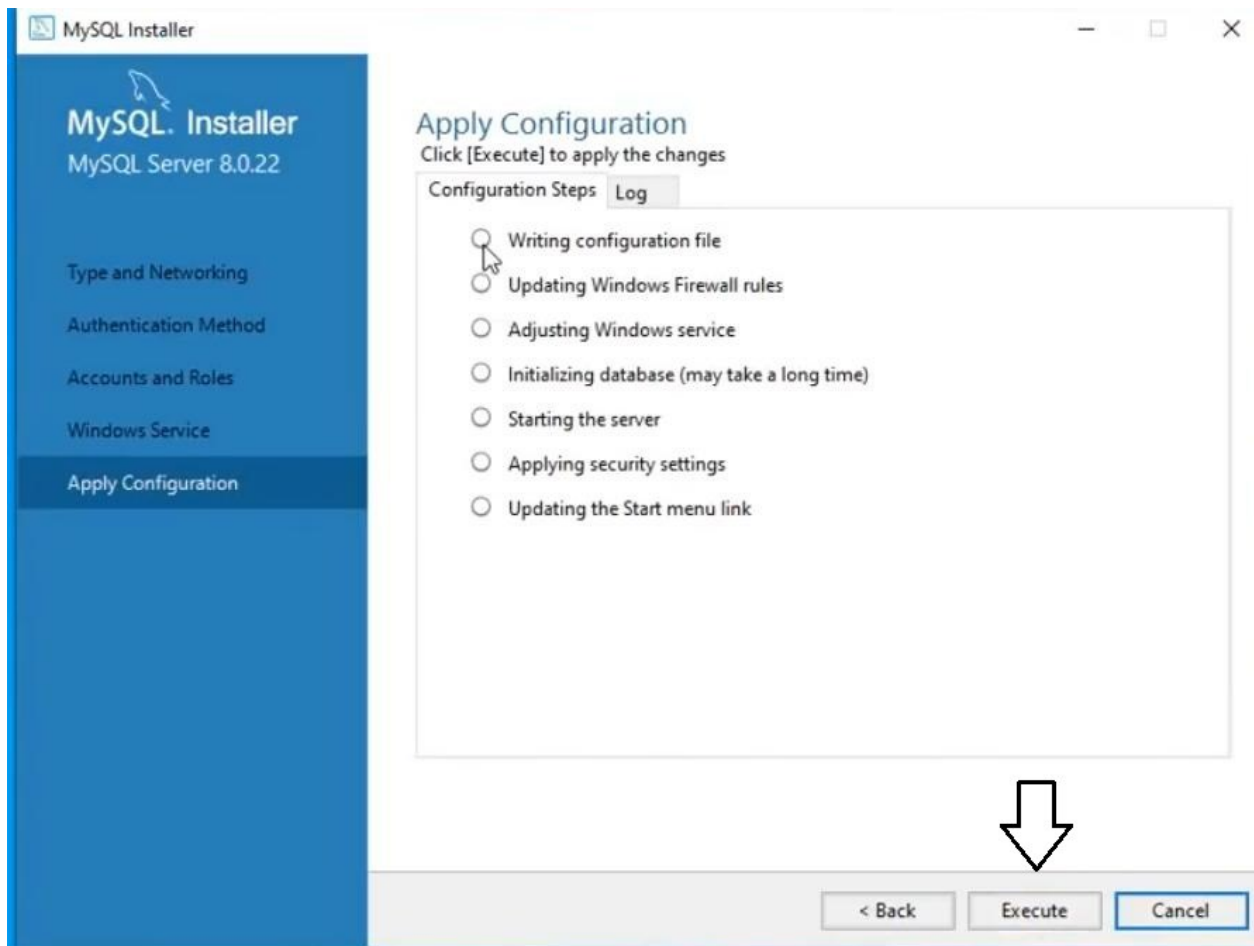
Step 19: In the configuration step, you will be prompted to configure the MySQL server. Set up a root password for your MySQL installation. Make sure to choose a strong password and remember it



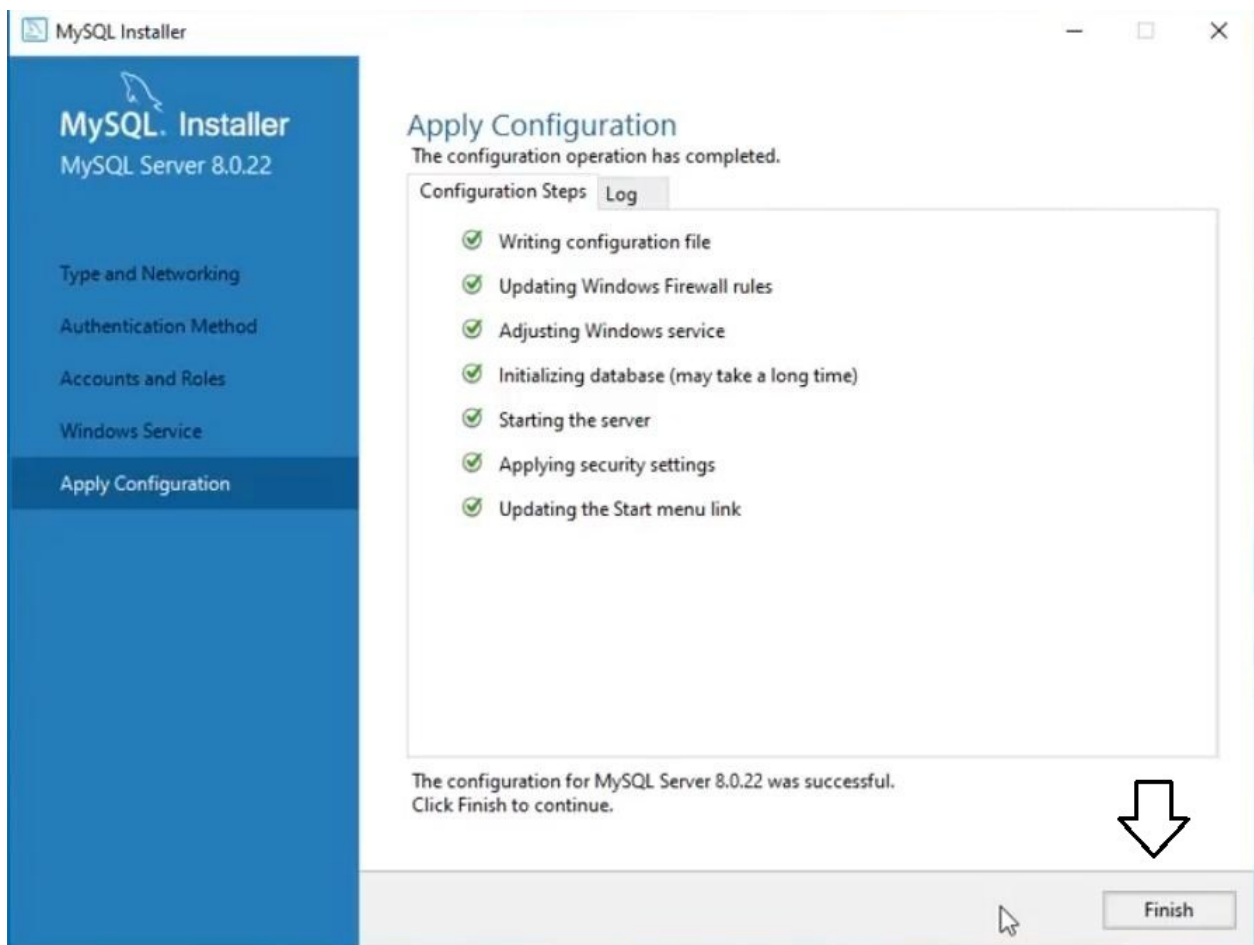
Step 20: Click on Next.



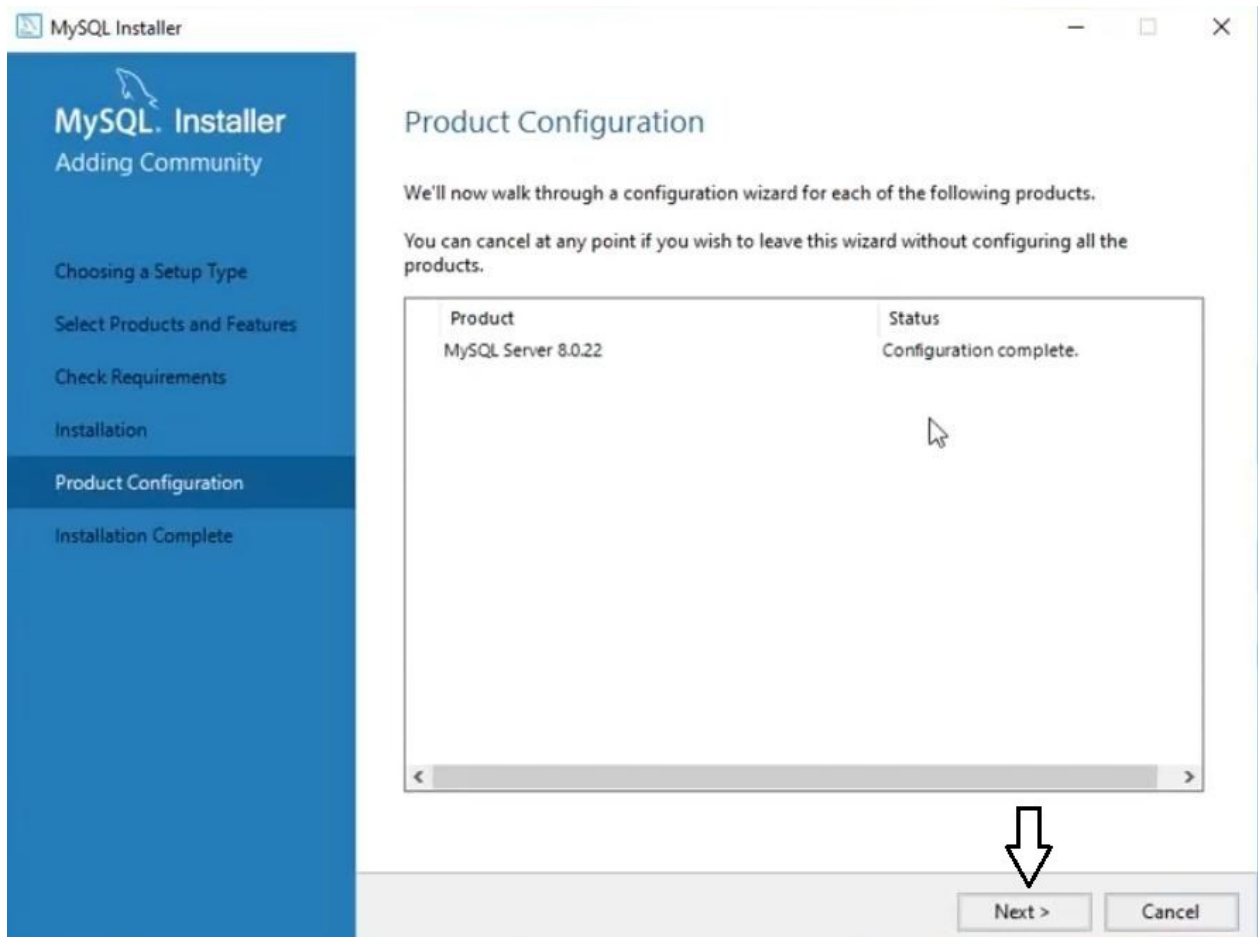
Step 21: Click on Execute.



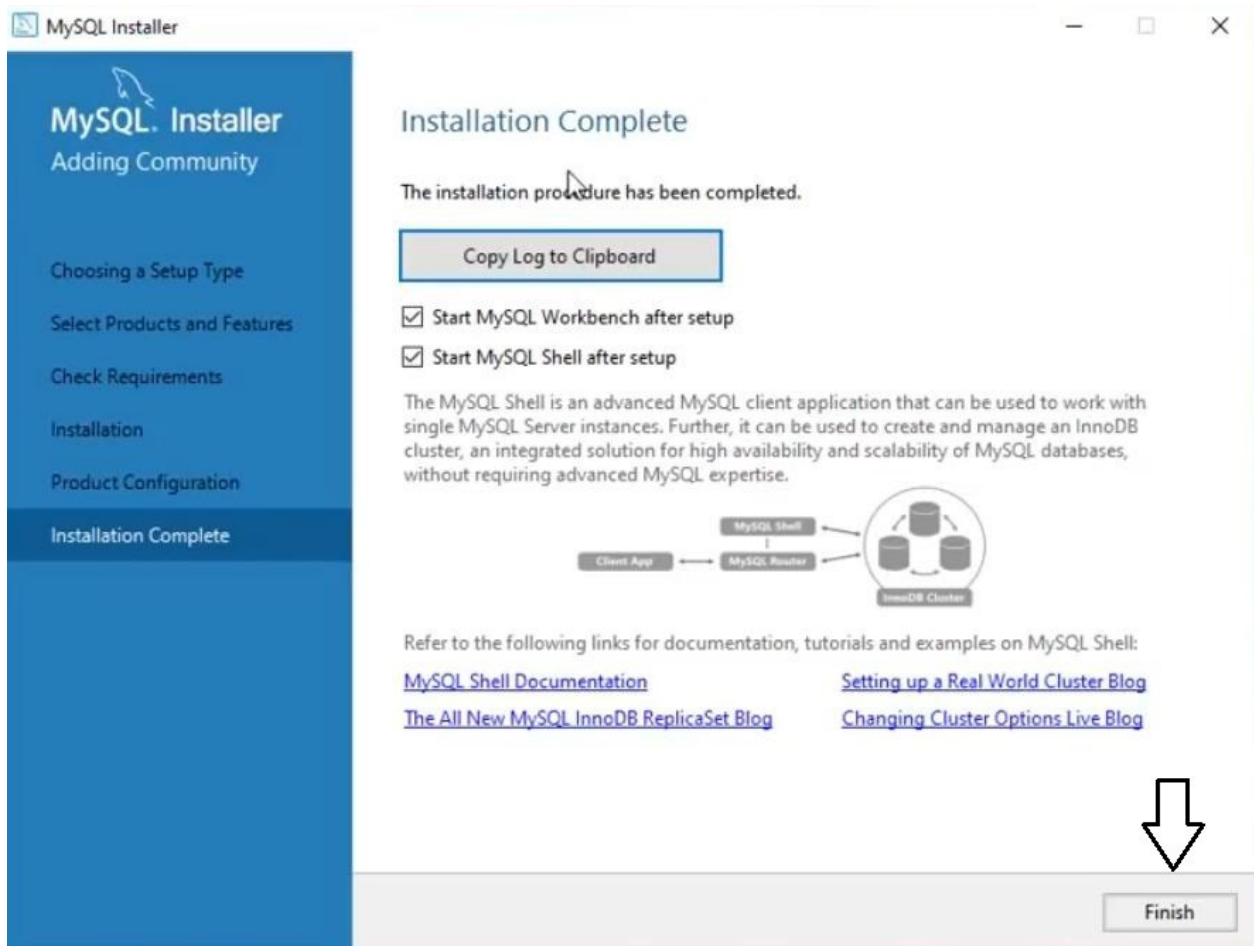
Step 22: Click Finish to complete the MySQL Workbench installation.



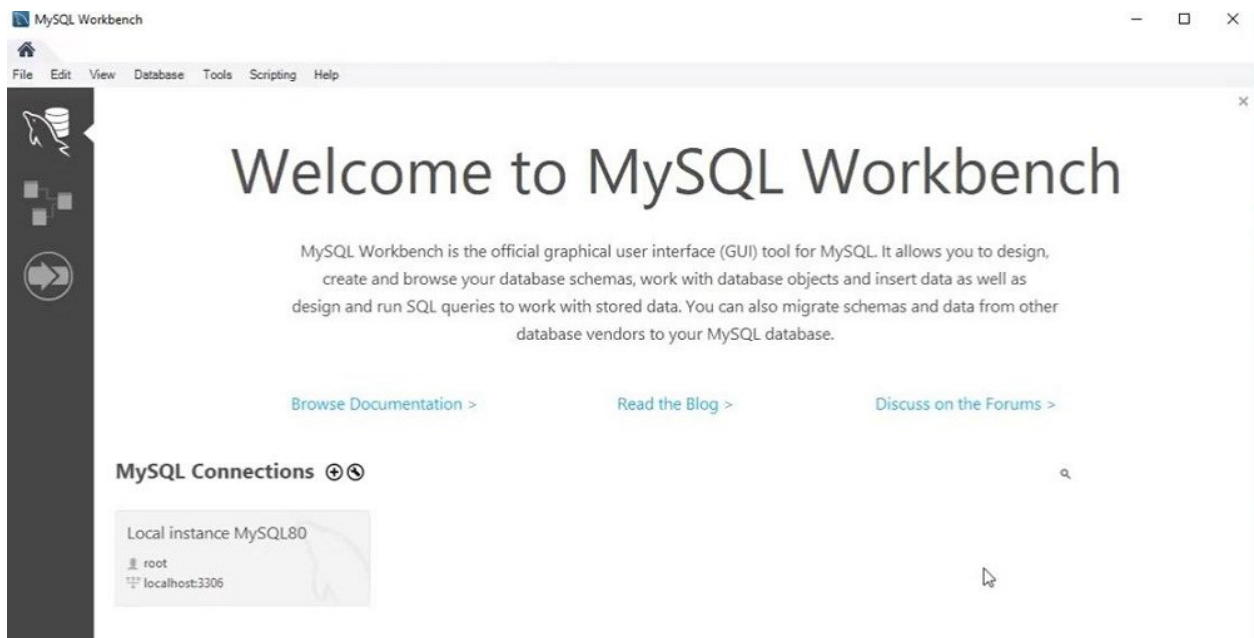
Step 23: Click on Next.



Step 24: Click on Finish.



Step 25: After the installation is complete, MySQL Workbench will automatically launch. If it doesn't open automatically, you can find it in the Start Menu by searching for "MySQL Workbench."



Hence, Your installation is successful.